

ESP32 Bluetooth RC Controller Main Entry Point Documentation

Miguel Chip

February 12, 2026

Contents

1	Project Entry File: ESP32_BT_Controller.ino	4
1.1	Purpose	4
1.2	Included Modules	4
1.3	Setup Function	5
1.4	Main Execution Loop	5
2	Bluetooth Communication Layer	7
2.1	Purpose	7
2.2	Design Strategy	7
2.3	Header File: bluetooth.h	7
2.4	Implementation File: bluetooth.cpp	7
2.5	Global Object Access	8
2.6	Device Discovery	8
2.7	Why This Architecture Works	8
2.8	System Data Flow	8
3	Packet Definition Layer	8
3.1	Purpose	8
3.2	Design Rules	9
3.3	Header File: packets.h	9
3.3.1	RC State Packet	9
3.3.2	Union Wrapper	10
3.3.3	Event Packet	10
3.4	Telemetry Output Packets	10
3.4.1	PanelPacket	10
3.4.2	IndicatorPacket	11
3.4.3	PlotPacket	11
3.5	Input Telemetry Packet	11
3.6	I2C Sensor Packet	11
3.7	Global Packet Instances	12
3.8	Packet Size Constants	12

4	Bluetooth Receive Layer	12
4.1	Purpose	12
4.2	Responsibilities	12
4.3	Header File: <code>receiver.h</code>	13
4.4	Source File: <code>receiver.cpp</code>	13
4.4.1	Includes	13
4.4.2	Receive Buffer	14
4.4.3	Main Function: <code>handleBluetooth()</code>	14
4.4.4	Reading Incoming Bytes	14
4.4.5	Searching for Packets	15
4.4.6	Packet Extraction	15
4.4.7	Event Trigger	15
4.4.8	Buffer Cleanup	15
4.5	Why a Rolling Buffer is Critical	15
5	Telemetry Transmission Layer	16
5.1	Purpose	16
5.2	Packet Channels	16
5.3	Header File: <code>telemetry.h</code>	16
5.4	Source File: <code>telemetry.cpp</code>	17
5.4.1	Includes	17
5.4.2	Timing State	17
5.4.3	Transmission Intervals	17
5.4.4	Plot Metadata	17
5.5	Config Telemetry Packet	18
5.6	Periodic Telemetry Sender	18
5.6.1	Panel Telemetry	18
5.6.2	Indicator Telemetry	19
5.6.3	Plot Telemetry	19
5.7	Checksum Model	19
5.8	System Role	19
6	Control Layer (Hardware Output Subsystem)	19
6.1	Purpose	19
6.2	Header File: <code>control.h</code>	20
6.3	Hardware Architecture	20
6.3.1	Signal Categories	20
6.4	Pin Map Overview	21
6.5	PWM Configuration	21
6.6	Initialization Function	21
6.7	Main Control Loop	22
6.7.1	Stick Mapping	22
6.7.2	Servo Mapping	22
6.8	Knob Mapping	22
6.9	Switch Decoding	22
6.10	Event Handling	22
6.11	System Role	23

7	Input Layer (Local Sensor and Button Telemetry)	23
7.1	Purpose	23
7.2	Header File: <code>input.h</code>	23
7.3	Supported Inputs	24
7.3.1	Analog Inputs	24
7.3.2	Digital Inputs	24
7.4	Packet Format	24
7.5	Timing Control	24
7.6	Initialization	25
7.7	Main Update Loop	25
7.8	Data Acquisition	25
7.9	Checksum Generation	25
7.10	Transmission	26
7.11	System Integration	26
8	I2C Sensor Telemetry Layer	26
8.1	Purpose	26
8.2	Public Interface: <code>i2c_sensors.h</code>	26
8.3	I2C Bus Configuration	27
8.4	Supported Sensors	27
8.4.1	Temperature Sensor	27
8.4.2	IMU (Accelerometer)	27
8.5	I2C Telemetry Packet	28
8.6	Transmission Rate	28
8.7	Telemetry Sender	28
8.8	Packet Assembly	28
8.9	Checksum and Transmission	28
8.10	System Integration	29
9	Debug and Diagnostic Layer	29
9.1	Purpose	29
9.2	Header File: <code>debug.h</code>	29
9.3	Implementation File: <code>debug.cpp</code>	30
9.3.1	State Packet Printer	30
9.3.2	Event Packet Printer	30
9.3.3	Panel Telemetry Printer	30
9.3.4	Indicator Telemetry Printer	31
9.3.5	Plot Telemetry Printer	31
9.4	Why a Dedicated Debug Layer?	31

1 Project Entry File: ESP32_BT_Controller.ino

1.1 Purpose

This file is the **main entry point** of the ESP32 Bluetooth RC Controller system. It does not contain control logic directly. Instead, it **coordinates all subsystems**:

- Bluetooth communication
- Packet decoding
- Telemetry transmission
- Hardware output control
- Local input and I2C sensors

This is known as a **modular layered architecture**. Each module is independent and easier to maintain.

1.2 Included Modules

```
#include <Arduino.h>
#include "bluetooth.h"
#include "packets.h"
#include "telemetry.h"
#include "receiver.h"
#include "debug.h"
#include "control.h"
#include "input.h"
#include "i2c_sensors.h"
```

Component Description

File	Purpose
Arduino.h	Core ESP32 and Arduino functions
bluetooth.h	Bluetooth SPP and SerialBT setup
packets.h	RC data packet structures
telemetry.h	Telemetry transmission logic
receiver.h	Bluetooth RX and packet decoding
debug.h	Serial debugging helpers
control.h	Hardware outputs (PWM, motors, LEDs)
input.h	Local ESP32 inputs
i2c_sensors.h	Sensor bus over I2C

1.3 Setup Function

```
void setup() {  
    Serial.begin(115200);  
    delay(1000);  
    Serial.println("Booting...");  
    SerialBT.begin(DVICE_NAME);  
    Serial.println("ESP32 Bluetooth Receiver Ready.");  
    Serial.printf(  
        "Listening for State (%d) and Event (%d)\n",  
        STATE_PACKET_SIZE, EVENT_PACKET_SIZE  
    );  
    controlInit();  
    i2cInit();  
}
```

Operational Overview

- Initializes USB serial at 115200 baud.
- Starts Bluetooth SPP using DVICE_NAME.
- Displays diagnostic information.
- Initializes all hardware outputs.
- Initializes I2C sensors.

1.4 Main Execution Loop

Real-Time Control Engine

The `loop()` function is the real-time execution engine of the ESP32 RC controller. It runs continuously after `setup()` finishes, and it is executed thousands of times per second.

This function synchronizes:

- Incoming control data from the Android application
- Physical outputs (motors, servos, lights, buzzer)
- Telemetry feedback sent back to Android
- Local user inputs connected to the ESP32
- I2C sensors such as voltage, current, or temperature monitors

Each task is isolated in its own function to respect the principle of **separation of concerns**. No function blocks the system using delays, so the controller remains responsive at all times.

Source Code

```
void loop() {  
    handleBluetooth();  
    sendTelemetryIfDue();  
    controlUpdate();  
    inputUpdate();  
    sendI2CTelemetry();  
}
```

Line-by-Line Description

1. `handleBluetooth();`

Reads incoming bytes from the Bluetooth SPP link. This function reconstructs data packets sent by the Android app, validates their checksum, identifies whether they are state packets or event packets, and updates the global structures defined in `packets.h`.

2. `sendTelemetryIfDue();`

Sends telemetry packets back to the Android app only when the defined time intervals have elapsed. This prevents flooding the Bluetooth channel while still keeping the interface responsive.

3. `controlUpdate();` Converts stick positions, knob values, and switch states into real hardware outputs. It updates PWM channels for motors and servos, and sets digital outputs for lights, relays, and auxiliary devices.

4. `inputUpdate();` Reads local inputs such as physical buttons, potentiometers, or digital sensors connected to the ESP32. When a change is detected, an event is sent to the Android application.

5. `sendI2CTelemetry();` Reads all connected I2C sensors and packages their values into telemetry frames that are transmitted to the Android app.

Execution Order Justification

1. Receive control data first
2. Transmit telemetry second
3. Update hardware outputs
4. Read local inputs
5. Send sensor telemetry

This ensures deterministic behavior, fast response, and safe motor control.

2 Bluetooth Communication Layer

2.1 Purpose

The Bluetooth layer provides the wireless communication link between the ESP32 controller and the Android application using the **Serial Port Profile (SPP)**.

This module:

- Creates a global Bluetooth serial interface
- Defines the Bluetooth device name
- Makes the Bluetooth object accessible to all modules

It does not contain logic or data processing. It only provides the communication channel.

2.2 Design Strategy

Instead of creating multiple Bluetooth objects, this system uses a **single global BluetoothSerial instance** shared across all modules.

This is achieved using the `extern` keyword in the header file and a single definition in the implementation file.

2.3 Header File: `bluetooth.h`

```
#ifndef BLUETOOTH_H
#define BLUETOOTH_H

#include <BluetoothSerial.h>

extern BluetoothSerial SerialBT;

#define DEVICE_NAME "ESP32-BT"

#endif
```

2.4 Implementation File: `bluetooth.cpp`

```
#include "bluetooth.h"

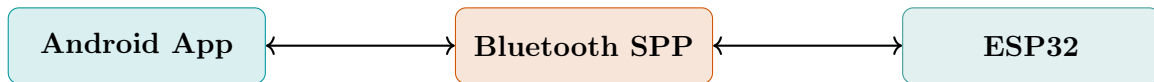
BluetoothSerial SerialBT;
```

2.5 Global Object Access

Any module that includes `bluetooth.h` can use:

```
SerialBT.begin (DEVICE_NAME) ;  
SerialBT.available () ;  
SerialBT.read () ;  
SerialBT.write (... ) ;  
SerialBT.hasClient () ;
```

This allows seamless communication between:



2.6 Device Discovery

The ESP32 advertises itself with the name:

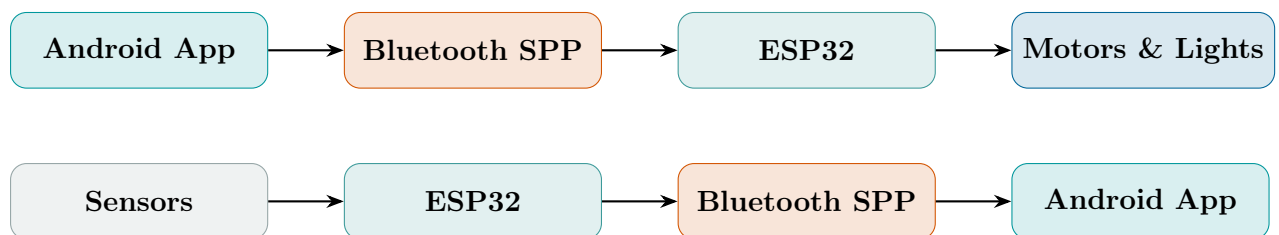
`ESP32-BT`

This name appears in the Android Bluetooth device list and identifies your RC controller.

2.7 Why This Architecture Works

- Prevents multiple Bluetooth stack instances
- Avoids memory duplication
- Allows clean modular design
- Keeps Bluetooth independent from logic layers

2.8 System Data Flow



3 Packet Definition Layer

3.1 Purpose

The packet layer defines every binary message exchanged between the Android application and the ESP32 controller.

This layer is the **single source of truth** for all communication structures and sizes. All other modules (`receiver.cpp`, `telemetry.cpp`, `control.cpp`, etc.) depend on the data types declared here.

The packet system is designed with the following goals:

- Fixed-size frames
- Low overhead for Bluetooth SPP
- Deterministic parsing
- Minimal RAM usage
- Fast checksum verification

3.2 Design Rules

All packets in this system follow the same protocol rules:

- Every packet begins with two header bytes
- Every packet ends with a checksum byte
- All structures are packed to avoid padding
- Union wrappers allow raw byte access or structured access
- All packets are defined in a single header file

3.3 Header File: `packets.h`

3.3.1 RC State Packet

This is the main control packet sent continuously by the Android application.

```
typedef struct __attribute__((packed)) {  
    byte startByte1, startByte2;  
    uint16_t leftStickX, leftStickY;  
    uint16_t rightStickX, rightStickY;  
    uint16_t leftKnob, rightKnob;  
    byte switches;  
    byte checksum;  
    byte endByte1, endByte2;  
} RcPacket;
```

Fields

- `startByte1`, `startByte2` — fixed header (synchronization)
- `leftStickX`, `leftStickY` — steering and throttle
- `rightStickX`, `rightStickY` — auxiliary pan and secondary motor
- `leftKnob`, `rightKnob` — analog controls for lights and buzzer
- `switches` — bit-packed digital switches
- `checksum` — additive checksum
- `endByte1`, `endByte2` — frame delimiter

3.3.2 Union Wrapper

```
typedef union {  
    RcPacket data;  
    byte bytes[sizeof(RcPacket)];  
} StatePacketUnion;
```

This allows:

- `rcStatePacket.data.leftStickX`
- `rcStatePacket.bytes[i]`

3.3.3 Event Packet

Event packets are sent only when a button is pressed.

```
typedef struct __attribute__((packed)) {  
    byte startByte1, startByte2;  
    byte eventId;  
    byte checksum;  
} EventPacket;
```

3.4 Telemetry Output Packets

3.4.1 PanelPacket

```
typedef struct __attribute__((packed)) {  
    byte header1, header2;  
    uint16_t leftPanelValue, rightPanelValue;  
    byte panelStates;  
    byte checksum;  
} PanelPacket;
```

Used to transmit dashboard gauge values.

3.4.2 IndicatorPacket

```
typedef struct __attribute__((packed)) {  
    byte header1, header2;  
    byte analogValue, batteryLevel;  
    byte checksum;  
} IndicatorPacket;
```

Used for small indicator widgets.

3.4.3 PlotPacket

```
typedef struct __attribute__((packed)) {  
    byte header1, header2;  
    byte numPlots;  
    byte plotValue1, plotValue2, plotValue3;  
    byte checksum;  
} PlotPacket;
```

Used for real-time signal plots.

3.5 Input Telemetry Packet

This packet reports physical inputs connected directly to the ESP32.

```
typedef struct {  
    byte h1;  
    byte h2;  
    uint16_t a34;  
    uint16_t a35;  
    uint16_t a36;  
    uint16_t a39;  
    byte d0;  
    byte d16;  
    byte d17;  
    byte checksum;  
} InputPacket;
```

Analog channels come from GPIO34, 35, 36, and 39. Digital inputs come from GPIO0, 16, and 17.

3.6 I2C Sensor Packet

```
typedef struct {  
    byte h1;  
    byte h2;  
    uint16_t temp;  
    uint16_t ax;  
    uint16_t ay;
```

```

    uint16_t az;
    byte sensorFlags;
    byte checksum;
} I2CPacket;

```

Used for sensors such as temperature probes and IMUs.

3.7 Global Packet Instances

Because header files must not allocate memory, all global packets are defined inside `packets.cpp`.

```

StatePacketUnion rcStatePacket;
EventPacketUnion rcEventPacket;
PanelPacket panelPacket;
IndicatorPacket indicatorPacket;
PlotPacket plotPacket;
InputPacket inputPacket;
I2CPacket i2cPacket;

```

3.8 Packet Size Constants

Each packet has a constant size exported for validation and parsing:

```

const int STATE_PACKET_SIZE;
const int EVENT_PACKET_SIZE;
const int PANEL_PACKET_SIZE;
const int INDICATOR_PACKET_SIZE;
const int PLOT_PACKET_SIZE;

```

4 Bluetooth Receive Layer

4.1 Purpose

The Bluetooth Receive Layer is responsible for converting a continuous, unstructured stream of Bluetooth bytes into valid control packets.

This module is the **front gate** of the system. Every command from the Android app must pass through this layer before it can affect motors, servos, LEDs, or telemetry.

4.2 Responsibilities

The receiver subsystem performs the following tasks:

- Read raw bytes from Bluetooth SPP
- Store them in a rolling buffer

- Search for valid packet headers
- Extract full packets
- Copy bytes into structured unions
- Call debug visualization
- Trigger control events

4.3 Header File: `receiver.h`

```
#ifndef RECEIVER_H
#define RECEIVER_H
void handleBluetooth();
#endif
```

Explanation

- Declares a single public function: `handleBluetooth()`
- This function is called continuously from the main loop
- It does not return data; it updates global packet structures

4.4 Source File: `receiver.cpp`

4.4.1 Includes

```
#include <Arduino.h>
#include "bluetooth.h"
#include "packets.h"
#include "receiver.h"
#include "debug.h"
#include "control.h"
```

These headers provide:

- Bluetooth stream (`SerialBT`)
- Packet definitions
- Debug printers
- Event control hooks

4.4.2 Receive Buffer

```
#define RX_BUFFER_SIZE 64
```

This buffer is intentionally larger than any packet. It allows the system to survive:

- Fragmented packets
- Multiple packets in one burst
- Noise or misaligned streams

4.4.3 Main Function: `handleBluetooth()`

```
void handleBluetooth() {  
    static byte buffer[RX_BUFFER_SIZE];  
    static int bytesRead = 0;
```

Why static?

- The buffer must persist between loop iterations
- Bluetooth packets may arrive slowly or in parts

4.4.4 Reading Incoming Bytes

```
while (SerialBT.available()) {  
    if (bytesRead < RX_BUFFER_SIZE) {  
        buffer[bytesRead++] = SerialBT.read();  
    } else {  
        memmove(buffer, buffer + 1, RX_BUFFER_SIZE - 1);  
        buffer[RX_BUFFER_SIZE - 1] = SerialBT.read();  
    }  
}
```

This logic:

- Appends new bytes until buffer is full
- If full, discards the oldest byte
- Always preserves the most recent stream

4.4.5 Searching for Packets

State Packet Detection

```
for (int i = 0; i <= bytesRead - 2; i++) {
    if (buffer[i] == 0xAA && buffer[i + 1] == 0x55) {
        if (bytesRead - i >= STATE_PACKET_SIZE) {
            packetStart = i;
            packetType = 1;
            break;
        }
    }
}
```

The loop scans the buffer for valid headers.

Event Packet Detection

```
if (buffer[i] == 0xBB && buffer[i + 1] == 0x66) {
    if (bytesRead - i >= EVENT_PACKET_SIZE) {
        packetStart = i;
        packetType = 2;
        break;
    }
}
```

4.4.6 Packet Extraction

```
memcpy(rcStatePacket.bytes, &buffer[packetStart], STATE_PACKET_SIZE);
```

This copies raw bytes into the union, instantly making structured fields available.

4.4.7 Event Trigger

```
controlHandleEvent(rcEventPacket.data.eventId);
```

This immediately pulses the hardware output defined in the control layer.

4.4.8 Buffer Cleanup

```
memmove(buffer, buffer + removeCount, bytesRead - removeCount);
bytesRead -= removeCount;
```

This removes the processed packet and shifts remaining bytes left.

4.5 Why a Rolling Buffer is Critical

Without a rolling buffer, Bluetooth fragmentation would corrupt packets. This design guarantees:

- No byte loss
- No desynchronization
- Continuous real-time control

5 Telemetry Transmission Layer

5.1 Purpose

The telemetry module is responsible for sending structured system data from the ESP32 to the Android application through Bluetooth.

Unlike the receiver module, this subsystem is **write-only**. It never parses incoming bytes. Its only job is to:

- Build telemetry packets
- Encode live system values
- Compute integrity checksums
- Send raw byte streams
- Regulate transmission timing

5.2 Packet Channels

Each telemetry message begins with the common header 0xCC and a packet type identifier:

Header	Type	Description
0xCC	0x11	PanelPacket
0xCC	0x22	IndicatorPacket
0xCC	0x33	PlotPacket
0xCC	0x44	ConfigPacket (variable length)

5.3 Header File: `telemetry.h`

```
#ifndef TELEMETRY_H
#define TELEMETRY_H
void sendTelemetryIfDue();
void sendConfigTelemetry();
#endif
```

Explanation

- `sendTelemetryIfDue()` sends periodic packets
- `sendConfigTelemetry()` sends variable-length configuration data

5.4 Source File: `telemetry.cpp`

5.4.1 Includes

```
#include <Arduino.h>
#include <math.h>
#include "bluetooth.h"
#include "packets.h"
#include "telemetry.h"
```

5.4.2 Timing State

```
static unsigned long lastPanelSend = 0;
static unsigned long lastIndicatorSend = 0;
static unsigned long lastPlotSend = 0;
```

Each timestamp stores the last transmission time for each packet type.

5.4.3 Transmission Intervals

```
const long panelInterval = 1000;
const long indicatorInterval = 250;
const long plotInterval = 50;
```

- Panel: once per second
- Indicator: four times per second
- Plot: twenty times per second

5.4.4 Plot Metadata

```
const char* plotNames[] = {"Volts", "Amps", "RPMs"};
const int NUM_PLOT_NAMES = 3;
```

These strings are transmitted to the Android UI so the plots can be labeled.

5.5 Config Telemetry Packet

```
void sendConfigTelemetry() {
    byte buf[128]; int idx=0;
    buf[idx++]=0xCC; buf[idx++]=0x44;
    buf[idx++]=NUM_PLOT_NAMES;
    for(int i=0;i<NUM_PLOT_NAMES;i++){
        byte l=strlen(plotNames[i]);
        buf[idx++]=l;
        memcpy(&buf[idx], plotNames[i], l);
        idx+=l;
    }
    if(SerialBT.hasClient()) SerialBT.write(buf,idx);
}
```

Explanation

- Header: 0xCC 0x44
- First payload byte = number of plot names
- Each plot name is sent as: [length] [characters]
- The packet size is dynamic

5.6 Periodic Telemetry Sender

```
void sendTelemetryIfDue() {
    unsigned long t=millis();
    if(!SerialBT.hasClient()) return;
}
```

Stops transmission if no Bluetooth client is connected.

5.6.1 Panel Telemetry

```
panelPacket={0xCC,0x11,1234,5678,0,0};
byte c=0;
for(int i=2;i<PANEL_PACKET_SIZE-1;i++)
    c+=((byte*)&panelPacket)[i];
panelPacket.checksum=c;
SerialBT.write((byte*)&panelPacket,PANEL_PACKET_SIZE);
```

Fields

- Left panel value
- Right panel value
- Panel states
- Additive checksum

5.6.2 Indicator Telemetry

```
indicatorPacket={0xCC,0x22,75,98,0};
byte c=0;
for(int i=2;i<INDICATOR_PACKET_SIZE-1;i++)
c+=((byte*)&indicatorPacket)[i];
indicatorPacket.checksum=c;
SerialBT.write((byte*)&indicatorPacket,INDICATOR_PACKET_SIZE);
```

5.6.3 Plot Telemetry

```
float s=t/1000.0f;
plotPacket={0xCC,0x33,3,
  (byte)((sin(s)*0.4)+0.6)*255,
  (byte)((cos(s)*0.2)+0.25)*255,
  (byte)((cos(s)*0.2)+0.5)*255,0};
```

These values generate live waveforms for UI testing.

5.7 Checksum Model

All telemetry packets use a simple additive checksum:

```
for(int i=2;i<PACKET_SIZE-1;i++)
checksum += rawBytes[i];
```

This verifies payload integrity without CPU cost.

5.8 System Role

The telemetry layer provides the Android app with:

- Real-time gauges
- Indicator status
- Plot graphs
- UI configuration

6 Control Layer (Hardware Output Subsystem)

6.1 Purpose

The control layer is the physical interface between the digital RC data received from the Android application and the real-world hardware connected to the ESP32.

This module translates:

- Stick positions
- Knob values
- Switch states
- Event commands

into electrical control signals such as:

- PWM for motors, servos, LEDs, and buzzers
- Digital HIGH/LOW outputs for relays, MOSFETs, and logic modules

The control layer is fully stateless and reactive: it does not store modes or logic. It only applies the current RC state to the hardware.

6.2 Header File: `control.h`

```
#ifndef CONTROL_H
#define CONTROL_H

#include <Arduino.h>

// Initialize all hardware outputs
void controlInit();

// Called every loop to apply RC state to hardware
void controlUpdate();

// Called when an EventPacket is received
void controlHandleEvent(byte eventId);

#endif
```

6.3 Hardware Architecture

6.3.1 Signal Categories

- **PWM outputs:** used for continuous control such as motors, servos, dimmers, and sound.
- **Digital outputs:** used for ON/OFF switches, relays, lights, and triggers.
- **Event outputs:** short pulses triggered by one-shot commands.

6.4 Pin Map Overview

Function	GPIO	Description
PWM Outputs		
Steering Servo	18	PWM steering control
Left Motor	19	PWM motor speed
Camera Pan	17	Auxiliary PWM servo
Right Motor	16	PWM motor speed
LED Dimmer	23	PWM brightness
Buzzer	27	PWM sound output
Digital Switches		
Switch 1	32	Digital output
Switch 2	33	Digital output
Switch 3	25	Digital output
Switch 4	26	Digital output
Switch 5	4	Digital output
Switch 6	5	Digital output
Switch 7	12	Digital output
Switch 8	13	Digital output
Event Outputs		
Event 1	14	Pulse output
Event 2	15	Pulse output
Event 3	2	Onboard LED
Event 4	0	Boot strap pin (caution)

6.5 PWM Configuration

Output Type	Frequency	Resolution
Servo	50 Hz	16 bits
Motor	20 kHz	8 bits
LED	5 kHz	8 bits
Buzzer	4 kHz	8 bits

6.6 Initialization Function

```
void controlInit() {  
    pinMode(PIN_SW1, OUTPUT);  
}
```

```

...
pinMode(PIN_EVT4, OUTPUT);

ledcAttach(PIN_STEER,    SERVO_FREQ, SERVO_RES);
ledcAttach(PIN_MOTOR_L, MOTOR_FREQ, MOTOR_RES);
ledcAttach(PIN_PAN,     SERVO_FREQ, SERVO_RES);
ledcAttach(PIN_MOTOR_R, MOTOR_FREQ, MOTOR_RES);
ledcAttach(PIN_LED,     LED_FREQ,   LED_RES);
ledcAttach(PIN_BUZZER,  BUZZ_FREQ,  BUZZ_RES);
}

```

This function prepares every GPIO pin and configures all PWM channels.

6.7 Main Control Loop

6.7.1 Stick Mapping

```

uint32_t steerPWM = mapServo(rcStatePacket.data.leftStickX);
uint32_t motorL   = mapServo(rcStatePacket.data.leftStickY);
uint32_t motorR   = mapServo(rcStatePacket.data.rightStickY);
uint32_t panPWM   = mapServo(rcStatePacket.data.rightStickX);

```

6.7.2 Servo Mapping

```

map(v, 0, 4095, 3277, 6553);

```

Converts joystick values into servo-safe pulse widths.

6.8 Knob Mapping

```

uint32_t ledPWM   = map(leftKnob, 0, 4095, 0, 255);
uint32_t buzzPWM = map(rightKnob, 0, 4095, 0, 255);

```

6.9 Switch Decoding

Each bit in the switch byte maps to a digital pin.

```

digitalWrite(PIN_SW1, sw & (1 << 0));
...
digitalWrite(PIN_SW8, sw & (1 << 7));

```

6.10 Event Handling

```

void controlHandleEvent(byte eventId) {
    switch (eventId) {
        case 0x01: pulse(PIN_EVT1); break;
    }
}

```

```

    case 0x02: pulse(PIN_EVT2); break;
    case 0x03: pulse(PIN_EVT3); break;
    case 0x04: pulse(PIN_EVT4); break;

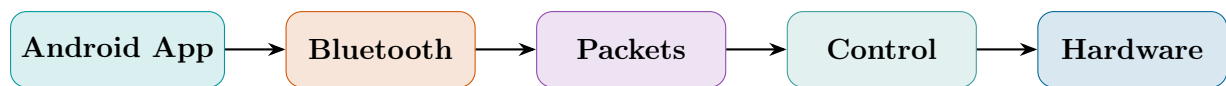
    default: break;
}
}

```

Each event triggers a short hardware pulse.

6.11 System Role

The control layer completes the full RC signal chain:



7 Input Layer (Local Sensor and Button Telemetry)

7.1 Purpose

The input layer allows the ESP32 to act not only as a receiver of commands, but also as a **data source** for the Android application.

This module:

- Reads local analog sensors
- Reads digital buttons and switches
- Packages the data into a binary telemetry frame
- Sends the data over Bluetooth to Android

It forms a bidirectional communication channel where both the phone and the ESP32 can generate live telemetry.

7.2 Header File: `input.h`

```

#ifndef INPUT_H
#define INPUT_H
#include <Arduino.h>

void inputInit();
void inputUpdate();

#endif

```

7.3 Supported Inputs

7.3.1 Analog Inputs

The ESP32 supports ADC channels on input-only pins:

GPIO	ADC Channel	Typical Use
ADC Inputs		
34	ADC1_6	Potentiometer, sensor
35	ADC1_7	Potentiometer, sensor
36	ADC1_0	Battery divider
39	ADC1_3	Pressure, temperature

7.3.2 Digital Inputs

GPIO	Usage
Local Buttons	
0	Push button / mode select
16	Push button
17	Push button

All digital pins use INPUT_PULLUP mode and therefore read LOW when pressed.

7.4 Packet Format

The input telemetry is packed into the following binary structure:

```
typedef struct {  
    byte h1;        // 0xCC  
    byte h2;        // 0x55  
    uint16_t a34;  
    uint16_t a35;  
    uint16_t a36;  
    uint16_t a39;  
    byte d0;  
    byte d16;  
    byte d17;  
    byte checksum;  
} InputPacket;
```

7.5 Timing Control

The packet is sent every:

```
const unsigned long inputInterval = 50; // ms
```

This creates a 20 Hz telemetry stream.

7.6 Initialization

```
void inputInit() {  
    pinMode(PIN_D0, INPUT_PULLUP);  
    pinMode(PIN_D16, INPUT_PULLUP);  
    pinMode(PIN_D17, INPUT_PULLUP);  
  
    pinMode(PIN_A34, INPUT);  
    pinMode(PIN_A35, INPUT);  
    pinMode(PIN_A36, INPUT);  
    pinMode(PIN_A39, INPUT);  
}
```

7.7 Main Update Loop

```
void inputUpdate() {  
    if (!SerialBT.hasClient()) return;  
    if (millis() - lastSend < inputInterval) return;  
    lastSend = millis();  
}
```

This ensures:

- No transmission when no Android device is connected
- Fixed bandwidth usage

7.8 Data Acquisition

```
inputPacket.a34 = analogRead(PIN_A34);  
inputPacket.a35 = analogRead(PIN_A35);  
inputPacket.a36 = analogRead(PIN_A36);  
inputPacket.a39 = analogRead(PIN_A39);  
  
inputPacket.d0  = !digitalRead(PIN_D0);  
inputPacket.d16 = !digitalRead(PIN_D16);  
inputPacket.d17 = !digitalRead(PIN_D17);
```

7.9 Checksum Generation

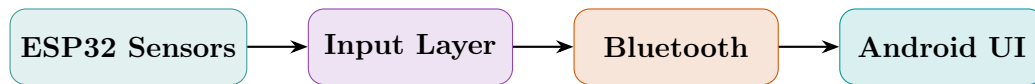
```
byte c = 0;  
for (int i = 2; i < INPUT_PACKET_SIZE - 1; i++)  
    c += ((byte*)&inputPacket)[i];  
  
inputPacket.checksum = c;
```

7.10 Transmission

```
SerialBT.write((byte*)&inputPacket, INPUT_PACKET_SIZE);
```

7.11 System Integration

The input layer completes the full telemetry loop:



This allows Android to display:

- Battery voltage
- External sensor data
- Physical button states

8 I2C Sensor Telemetry Layer

8.1 Purpose

The I2C sensor layer allows the ESP32 to read external digital sensors and stream their values in real time to the Android application using the same Bluetooth telemetry channel.

This module:

- Initializes the I2C bus
- Reads temperature sensors
- Reads inertial measurement units (IMU)
- Packs the sensor values into a telemetry frame
- Sends the packet to the Android app

The module is **write-only**: it never parses incoming data.

8.2 Public Interface: `i2c_sensors.h`

```
void i2cInit();  
void sendI2CTelemetry();
```

8.3 I2C Bus Configuration

```
void i2cInit() {  
    Wire.begin(21,22);  
}
```

- GPIO 21 → SDA
- GPIO 22 → SCL
- Standard I2C pull-up resistors are required

8.4 Supported Sensors

8.4.1 Temperature Sensor

- Address: 0x48
- 16-bit temperature register

```
uint16_t readTemp() {  
    Wire.beginTransaction(I2C_ADDR_TEMP);  
    Wire.write(0);  
    Wire.endTransmission();  
    Wire.requestFrom(I2C_ADDR_TEMP, 2);  
    return (Wire.read() << 8) | Wire.read();  
}
```

8.4.2 IMU (Accelerometer)

- Address: 0x68
- Registers: 0x3B–0x40 (AX, AY, AZ)

```
void readIMU(int16_t &ax, int16_t &ay, int16_t &az) {  
    Wire.beginTransaction(I2C_ADDR_IMU);  
    Wire.write(0x3B);  
    Wire.endTransmission(false);  
    Wire.requestFrom(I2C_ADDR_IMU, 6);  
    ax = (Wire.read() << 8) | Wire.read();  
    ay = (Wire.read() << 8) | Wire.read();  
    az = (Wire.read() << 8) | Wire.read();  
}
```

8.5 I2C Telemetry Packet

```
typedef struct {
    byte h1;        // 0xCC
    byte h2;        // 0x66
    uint16_t temp;
    uint16_t ax;
    uint16_t ay;
    uint16_t az;
    byte sensorFlags;
    byte checksum;
} I2CPacket;
```

8.6 Transmission Rate

```
const unsigned long i2cInterval = 100; // ms
```

This produces a 10 Hz telemetry stream.

8.7 Telemetry Sender

```
void sendI2CTelemetry() {
    if(!SerialBT.hasClient()) return;
    if(millis()-lastSend<i2cInterval) return;
    lastSend=millis();
}
```

8.8 Packet Assembly

```
i2cPacket.h1=0xCC;
i2cPacket.h2=0x66;
i2cPacket.temp = readTemp();
readIMU(ax,ay,az);

i2cPacket.ax=ax;
i2cPacket.ay=ay;
i2cPacket.az=az;
i2cPacket.sensorFlags=0b00000011;
```

8.9 Checksum and Transmission

```
byte c=0;
for(int i=2;i<I2C_PACKET_SIZE-1;i++)
    c+=((byte*)&i2cPacket)[i];

i2cPacket.checksum=c;
SerialBT.write((byte*)&i2cPacket,I2C_PACKET_SIZE);
```

8.10 System Integration



This enables live display of:

- Board temperature
- Vehicle acceleration
- Motion detection

9 Debug and Diagnostic Layer

9.1 Purpose

The debug layer centralizes all formatted Serial output used for monitoring, testing, and validating the communication system.

This module:

- Decodes binary packet fields into readable text
- Prints values in a clear, human-friendly format
- Keeps debug logic separate from control logic

It **never** communicates via Bluetooth and **never** modifies data. It only reads global packet structures.

9.2 Header File: `debug.h`

```
#ifndef DEBUG_H
#define DEBUG_H

#include <Arduino.h>
#include "packets.h"

void printStatePacket();
void printEventPacket();
void printPanelPacket();
void printIndicatorPacket();
void printPlotPacket();

#endif
```

9.3 Implementation File: `debug.cpp`

9.3.1 State Packet Printer

```
void printStatePacket() {
    Serial.println("\n--- RC STATE PACKET ---");

    Serial.printf("L Stick: X=%u Y=%u\n",
rcStatePacket.data.leftStickX,
rcStatePacket.data.leftStickY);

    Serial.printf("R Stick: X=%u Y=%u\n",
rcStatePacket.data.rightStickX,
rcStatePacket.data.rightStickY);

    Serial.printf("Knobs: L=%u R=%u\n",
rcStatePacket.data.leftKnob,
rcStatePacket.data.rightKnob);

    byte s = rcStatePacket.data.switches;
    Serial.printf("Switch Byte: 0x%02X\n", s);

    for (int i = 0; i < 6; i++) {
        Serial.printf(
            " S%d = %s\n",
            i + 1,
            (s & (1 << i)) ? "ON" : "OFF"
        );
    }

    Serial.printf("Checksum: %u\n", rcStatePacket.data.checksum);
    Serial.println("-----");
}
```

9.3.2 Event Packet Printer

```
void printEventPacket() {
    Serial.println("\n--- EVENT PACKET ---");
    Serial.printf("Event ID: 0x%02X\n", rcEventPacket.data.eventId);
    Serial.printf("Checksum: 0x%02X\n", rcEventPacket.data.checksum);
    Serial.println("-----");
}
```

9.3.3 Panel Telemetry Printer

```
void printPanelPacket() {
    Serial.println("\n--- PANEL TELEMETRY ---");
    Serial.printf("Left: %u Right: %u\n",
panelPacket.leftPanelValue,
```

```

    panelPacket.rightPanelValue);
    Serial.printf("States: 0x%02X\n", panelPacket.panelStates);
    Serial.printf("Checksum: %u\n", panelPacket.checksum);
}

```

9.3.4 Indicator Telemetry Printer

```

void printIndicatorPacket() {
    Serial.println("\n--- INDICATOR TELEMETRY ---");
    Serial.printf("Analog: %u\n", indicatorPacket.analogValue);
    Serial.printf("Battery: %u%%\n", indicatorPacket.batteryLevel);
    Serial.printf("Checksum: %u\n", indicatorPacket.checksum);
}

```

9.3.5 Plot Telemetry Printer

```

void printPlotPacket() {
    Serial.println("\n--- PLOT TELEMETRY ---");
    Serial.printf("Plots: %u\n", plotPacket.numPlots);
    Serial.printf("P1=%u P2=%u P3=%u\n",
        plotPacket.plotValue1,
        plotPacket.plotValue2,
        plotPacket.plotValue3);
    Serial.printf("Checksum: %u\n", plotPacket.checksum);
}

```

9.4 Why a Dedicated Debug Layer?

- Keeps control and telemetry code clean
- Allows easy enabling/disabling of Serial logs
- Prevents timing issues in Bluetooth logic
- Helps visualize binary data structures